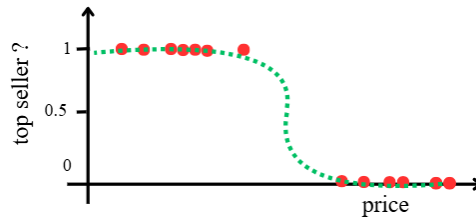


Machine Learning 2

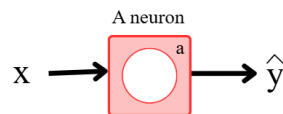
Example:

Consider a classification task of deciding whether an item would be a top seller or not based on selling price.



Typically we would have a hypothesis function, in a supervised setting (inputs-outputs) trying to learn this.

In the world of Neural Networks, this example could be represented as the following setup:

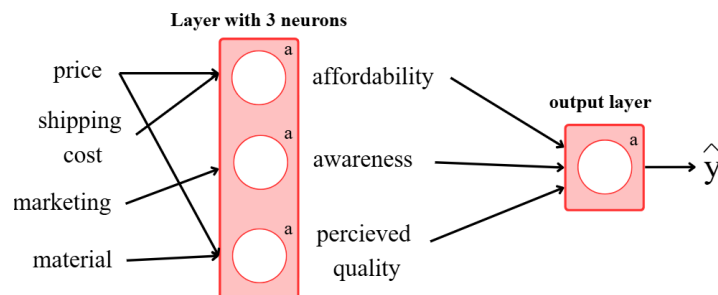


- $\mathbf{x}$: input
- $\hat{y}$: output
- $\mathbf{a}$: Activation function (hypothesis function in this context), which will output the probability of a product being a top-seller based on price alone.

Note: Our examples generally require multiple features, and based on combinations of those features we can ascertain different attributes, which can themselves become features for our output. This is most useful if the target output is a complex combination of the input features (the relationship of which may not be obvious like previous examples of Supervised Learning)

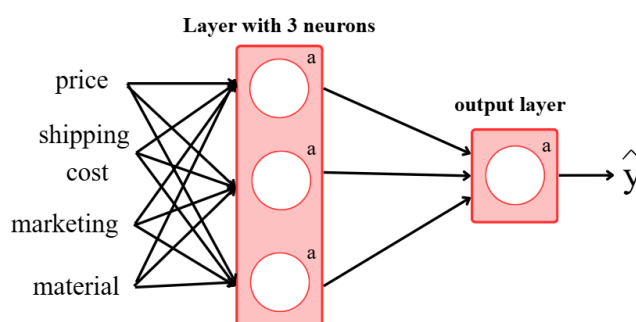
Example:

So, we can have an example of a model which is more complicated than a single neuron, here we have 4 inputs, 3 activation values and 1 output layer.



Note: In the above example we had to decide what was a function of what feature for the first layer but generally all features are connected to all neurons. We just expect them to learn to ignore what isn't important.

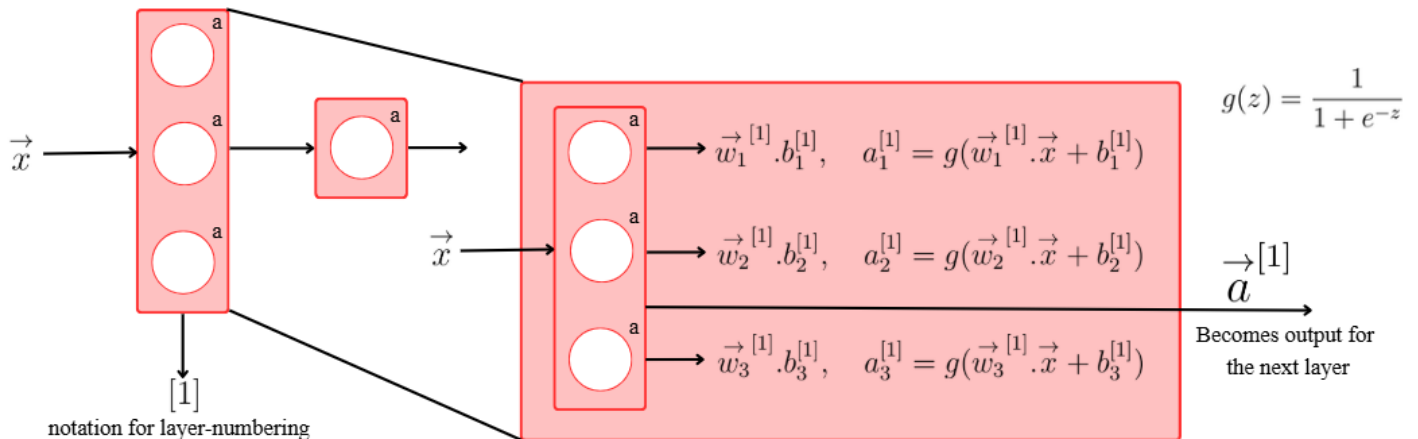
Example:



Note: This idea of ”**automatic-feature-selection**” is what makes Neural Networks so much better than previous paradigms of supervised learning which require a heavy amount of feature engineering/selection and then consequently diagnosis.

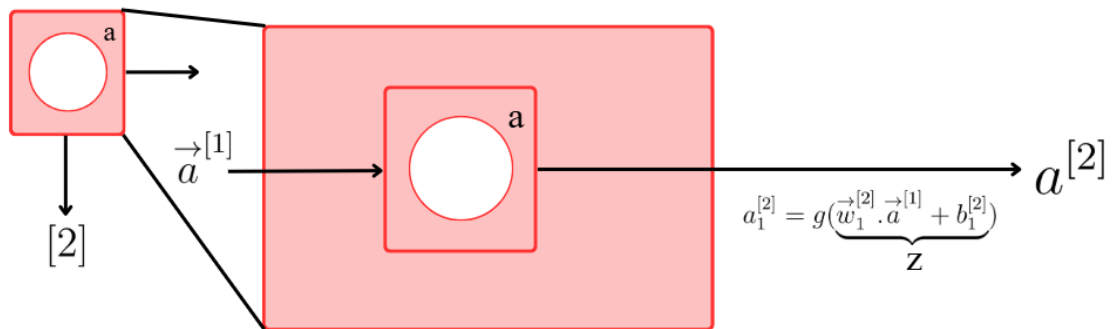
Example:

A microscopic look at the 1st layer of the previous example.



A microscopic look at the output layer of the previous example.

Layer - 2 / Output Layer

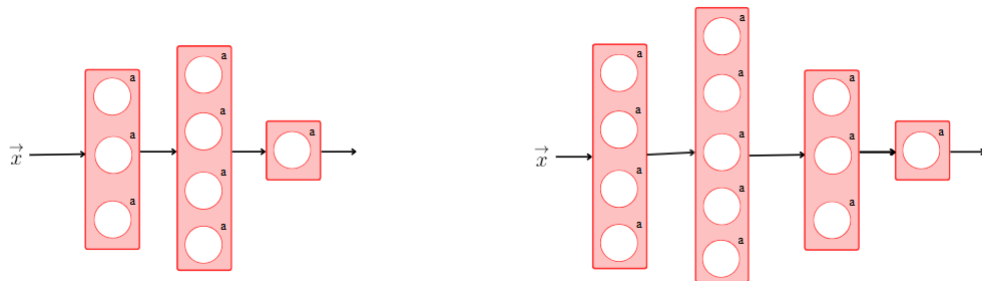


where,

$$\hat{y} = \begin{cases} 1 & \text{if } a^{[2]} \geq 0.5 \\ 0 & \text{if } a^{[2]} < 0.5 \end{cases}$$

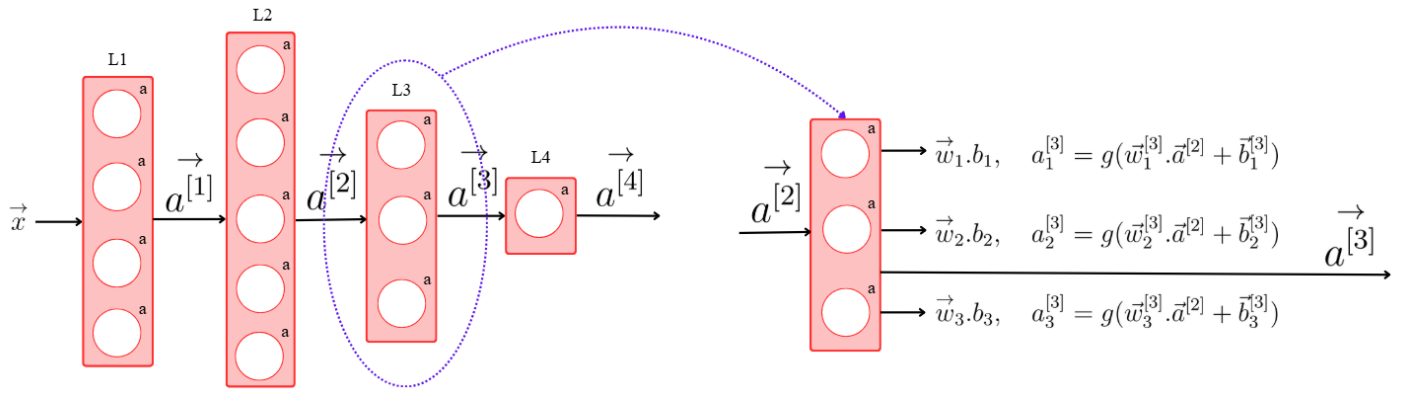
Example:

We can have Neural Networks with more than an input-output layer, The layers that are intermediate to these two are typically called ”**Hidden Layers**”



Example:

Let us consider an intermediate layer of a more complicated Neural Network, with 2 hidden layers



where,

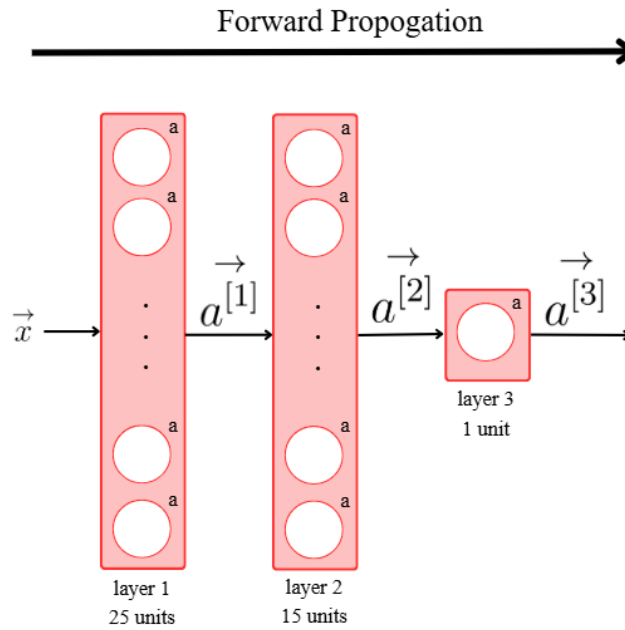
$$\vec{a}^{[3]} = \begin{bmatrix} a_1^{[3]} \\ a_2^{[3]} \\ a_3^{[3]} \end{bmatrix}$$

That is,

$$a_j^{[l]} = g(\vec{w}_j^{[l]} \cdot \vec{a}^{[l-1]} + b_j^{[l]})$$

Example:

handwritten Digit recognition, to check if the digit is 1 ? (binary: yes or no)



which decides based on the following decision boundary,

$$\hat{y} = \begin{cases} 1 & \text{if } a_1^{[3]} \geq 0.5 \\ 0 & \text{if } a_1^{[3]} < 0.5 \end{cases}$$

where the output of layer-1 produces,

$$\vec{a}^{[1]} = \begin{bmatrix} g(\vec{w}_1^{[1]} \cdot \vec{x} + b_1^{[1]}) \\ g(\vec{w}_2^{[1]} \cdot \vec{x} + b_2^{[1]}) \\ \vdots \\ g(\vec{w}_{25}^{[1]} \cdot \vec{x} + b_{25}^{[1]}) \end{bmatrix}$$

and the output of layer-2 looks like,

$$\vec{a}^{[2]} = \begin{bmatrix} g(\vec{w}_1^{[2]} \cdot \vec{a}^{[1]} + b_1^{[2]}) \\ g(\vec{w}_2^{[2]} \cdot \vec{a}^{[1]} + b_2^{[2]}) \\ \vdots \\ g(\vec{w}_{15}^{[2]} \cdot \vec{a}^{[1]} + b_{15}^{[2]}) \end{bmatrix}$$

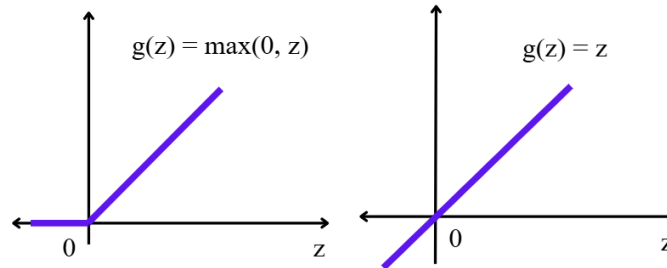
Alterations to the Sigmoid function

- A very common choice is **ReLU** (Rectified Linear Unit)

$$g(z) = \begin{cases} 0 & \text{if } z < 0 \\ z & \text{if } z \geq 0 \end{cases}$$

- **Linear Activation** function is another choice, though less common that would replace Sigmoid, where

$$g(z) = z = \vec{w} \cdot \vec{x} + b$$



Choosing Activation Functions

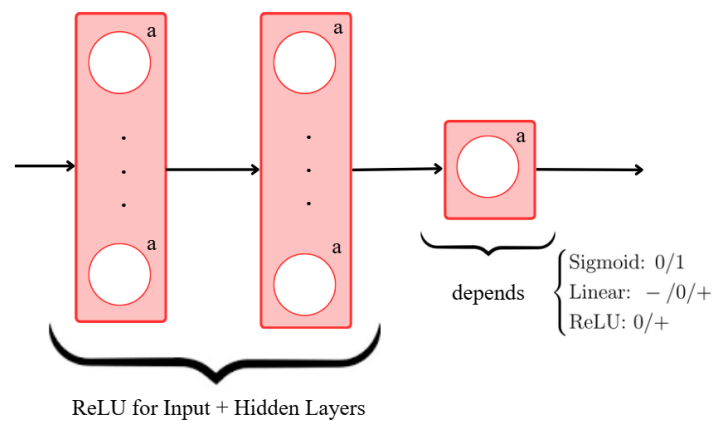
Output layer is determined based on problem:

- binary classification: **Sigmoid**
- regression: **Linear** (negative + positive values)
- regression: **ReLU** (non-negative)

What about hidden layers ?

- ReLU common choice
- ReLU learns faster than sigmoid (generally)

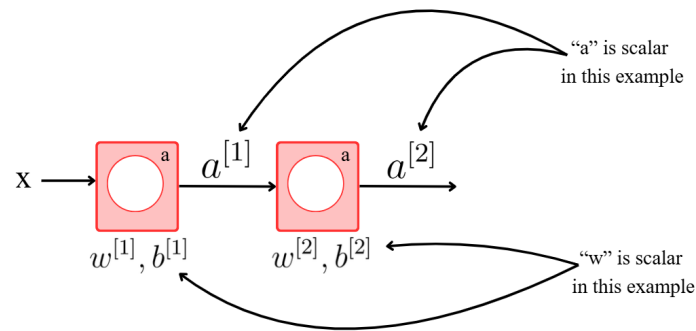
General Standard for Activation Functions



Why we need activation functions

Example:

Consider an example with a single feature and 2 linear activation functions, i.e., [2] (a scalar input effectively) and $a = g(z) = z$



then,

$$a^{[1]} = w_1^{[1]} \cdot x + b_1^{[1]} - (*)$$

$$a^{[2]} = w_1^{[2]} \cdot a^{[1]} + b_1^{[2]} - (**)$$

Using (*) and (**) we get,

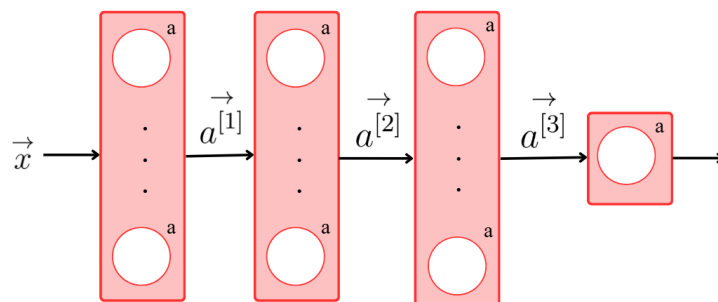
$$\begin{aligned} a^{[2]} &= w_1^{[2]} \cdot (w_1^{[1]} \cdot x + b_1^{[1]}) + b_1^{[2]} \\ \Rightarrow \vec{a}^{[2]} &= \underbrace{(\vec{w}_1^{[2]} \cdot \vec{w}_1^{[1]})}_w \cdot x + \underbrace{w_1^{[2]} \cdot b_1^{[1]} + b_1^{[2]}}_b \\ \Rightarrow \vec{a}^{[2]} &= w \cdot x + b \end{aligned}$$

Note: That is it was pointless to have 2 subsequent linear regressing neurons. We could have just used Linear Regression or a single neuron anyway. This is because the linear function of a linear function is linear,

$$f \circ f(x) = g(x)$$

where if $f(x)$ is linear, then so will be $g(x)$

General Case



Note: If all the activation functions, a are linear then this entire network irrespective of number of layers or neurons per layer is the same as performing **linear regression**.

Note: And if just the output activation function, a is sigmoid then its the same as performing **logistic regression**.

Multi class Classification

Example:

Given images of digits classify which digit is which,

0, 1, 2, 3, 4, 5, 6, 7, 8, 9

target "y" has more than 2 possible values, 10 to be precise

SoftMax Regression

Considered generalisation of logistic regression upto N classes.

Example:

Logistic Regression Setup:

$$z = \vec{w} \cdot \vec{x} + b$$

$$\{a_1 = g(x) = \frac{1}{1 + e^{-z}} = P(y = 1|\vec{x}) \quad \text{say } 0.71$$

$$\{a_2 = 1 - a_1 = P(y = 0|\vec{x}) \quad \text{say } 1 - 0.71 = 0.29$$

Consider 4 possible outputs: $y = 1, 2, 3, 4$,

$$- z_1 = \vec{w}_1 \cdot \vec{x} + b_1$$

$$a_1 = \frac{e^{z_1}}{e^{z_1} + e^{z_2} + e^{z_3} + e^{z_4}} = P(y = 1|\vec{x}) \quad \text{say } 0.71$$

$$- z_2 = \vec{w}_2 \cdot \vec{x} + b_2$$

$$a_2 = \frac{e^{z_2}}{e^{z_1} + e^{z_2} + e^{z_3} + e^{z_4}} = P(y = 2|\vec{x}) \quad \text{say } 0.20$$

$$- z_3 = \vec{w}_3 \cdot \vec{x} + b_3$$

$$a_3 = \frac{e^{z_3}}{e^{z_1} + e^{z_2} + e^{z_3} + e^{z_4}} = P(y = 3|\vec{x}) \quad \text{say } 0.15$$

$$- z_4 = \vec{w}_4 \cdot \vec{x} + b_4$$

$$a_4 = \frac{e^{z_4}}{e^{z_1} + e^{z_2} + e^{z_3} + e^{z_4}} = P(y = 4|\vec{x}) \quad \text{say } 0.35$$

Generalising (N possible outputs),

$$z_j = \vec{w}_j \cdot \vec{x} + b, \quad j = 1, \dots, N$$

parameters: $w_1, w_2, \dots, w_N$
 $b_1, b_2, \dots, b_N$

Summary

Logistic Regression

$$z = \vec{w} \cdot \vec{x} + b$$

$$a_1 = g(z) = \frac{1}{1 + e^{-z}} = P(y = 1 | \vec{x})$$

$$a_2 = 1 - a_1 = P(y = 0 | \vec{x})$$

$$\text{loss} = -y \log(a_1) - (1 - y) \log(1 - a_1)$$

$$J(\vec{w}, b) = \text{average loss}$$

Softmax Regression

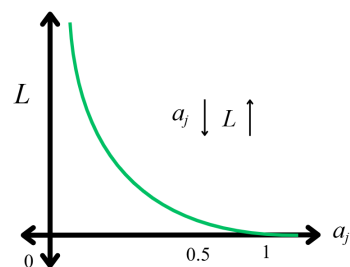
$$a_1 = \frac{e^{z_1}}{e^{z_1} + e^{z_2} + \dots + e^{z_N}} = P(y = 1 | \vec{x})$$

$\vdots$

$$a_N = \frac{e^{z_N}}{e^{z_1} + e^{z_2} + \dots + e^{z_N}} = P(y = N | \vec{x})$$

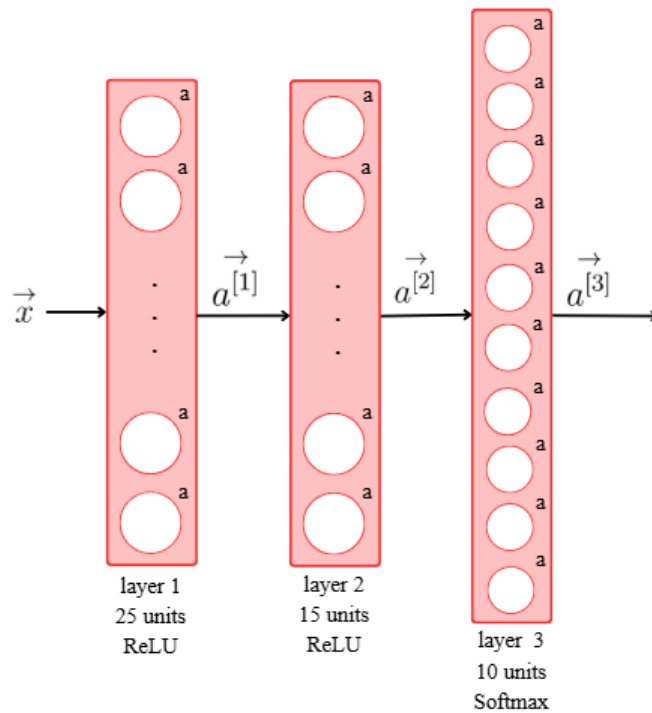
$$\text{loss}(a_1, \dots, a_N, y) = \begin{cases} -\log(a_1) & \text{if } y = 1 \\ -\log(a_2) & \text{if } y = 2 \\ \vdots & \\ -\log(a_N) & \text{if } y = N \end{cases}$$

$$\text{loss} = -\log(a_j) \quad \text{if } y = j$$



Neural Network with Soft Max Output

Example:



$$z_1^{[3]} = \vec{w}_1^{[3]} \cdot \vec{a}^{[2]} + b_1^{[5]}, \quad a_1^{[3]} = \frac{e^{z_1^{[3]}}}{e^{z_1^{[3]}} + e^{z_2^{[3]}} + \dots + e^{z_{10}^{[3]}}} = P(y = 1 | \vec{x})$$

$$\vdots$$

$$z_{10}^{[3]} = \vec{w}_{10}^{[3]} \cdot \vec{a}^{[2]} + b_{10}^{[5]}, \quad a_{10}^{[3]} = \frac{e^{z_{10}^{[3]}}}{e^{z_1^{[3]}} + e^{z_2^{[3]}} + \dots + e^{z_{10}^{[3]}}} = P(y = 0 | \vec{x})$$

In **Logistic Regression**,

$$\text{Individual functions:} \quad a_1^{[3]} = g(z_1^{[3]}), \quad a_2^{[3]} = g(z_2^{[3]}) \quad \dots$$

In **SoftMax**,

$$\text{Simultaneous functions:} \quad \vec{a}_1^{[3]} = (a_1^{[3]}, \dots, a_{10}^{[3]}) = g(z_1^{[3]}, \dots, z_{10}^{[3]})$$

Multi-Label Classification

hrule

Example:

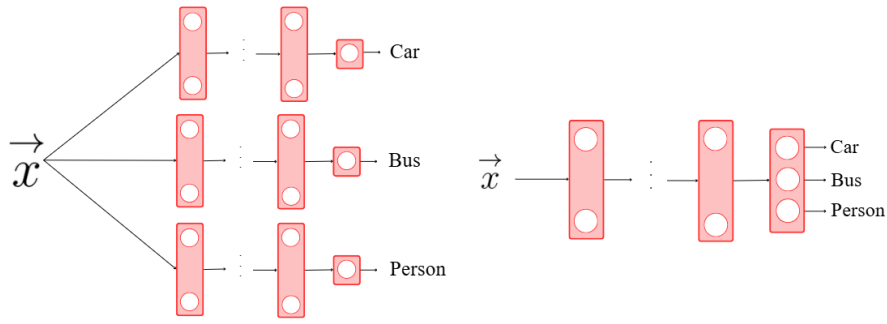
Photos that contain different elements !

$$examples = \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix}, \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}, \begin{bmatrix} 1 \\ 1 \\ 0 \end{bmatrix}$$

is there a car ?	Yes	No	Yes
is there a bus ?	No	No	Yes
is there a person ?	Yes	Yes	No

2 approaches:

- 3 separate ML problems, i.e., 3 Neural Nets and put it together.
- 1 Neural Net with simultaneous detection.

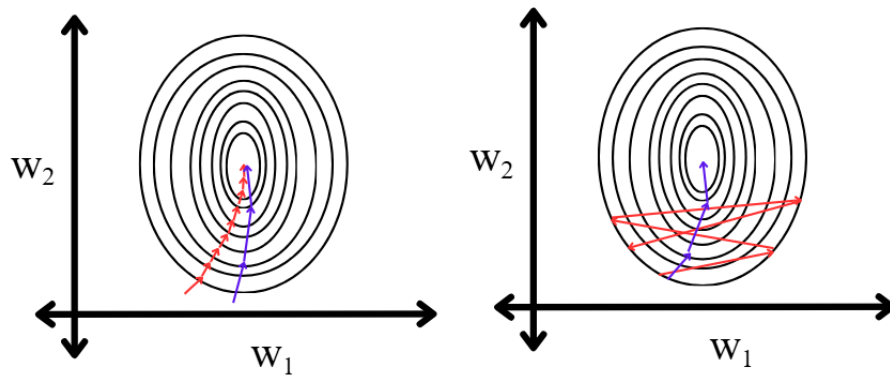


Note: Multi-class classification assigns each input to exactly one class out of several possible classes, such as classifying an image as either a cat, dog, or horse. **Multi-label classification** allows an input to belong to multiple classes at the same time, such as tagging a photo with both “beach” and “sunset”

ADAM

When performing updates in Gradient Descent, we can have 2 different needs,

- If w_j or b keeps moving in the direction, but slowly, we can go faster (learning rate).
- If w_j or b keeps oscillating between the two, we can reduce our speed (learning rate).



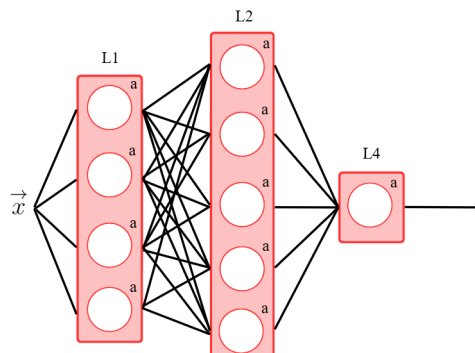
Note: ADAM: ADAPtive Movement Estimation

ADAM doesn't just use one global learning rate but has different learning rates for different models

$$\begin{cases} w_1 \triangleq w_1 - \alpha_1 \frac{\partial}{\partial w_1} J(\vec{w}, b) \\ \vdots \\ w_{10} \triangleq w_{10} - \alpha_{10} \frac{\partial}{\partial w_{10}} J(\vec{w}, b) \\ b \triangleq b - \alpha_{11} \frac{\partial}{\partial b} J(\vec{w}, b) \end{cases}$$

Types of Layers

- **Dense:** When all outputs of previous layer connects to all inputs of the subsequent layers

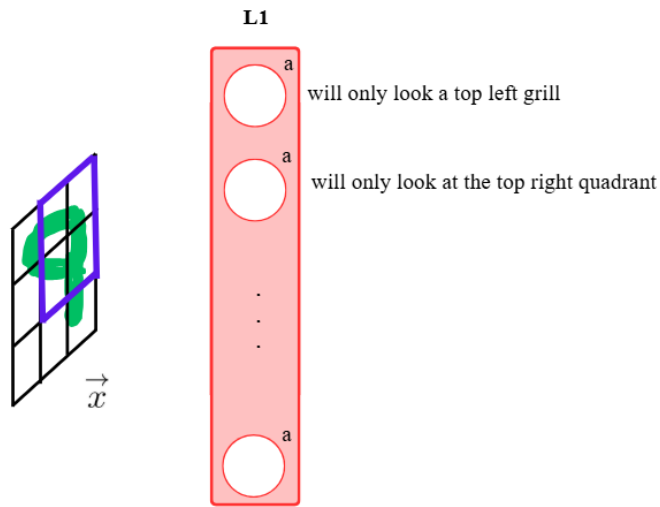


More formally, each neuron output is a function of all the activation outputs of the previous layer,

$$\vec{a}_1^{[2]} = g(\vec{w}_1^{[2]} \cdot \vec{a}^{[1]} + b_1^{[2]})$$

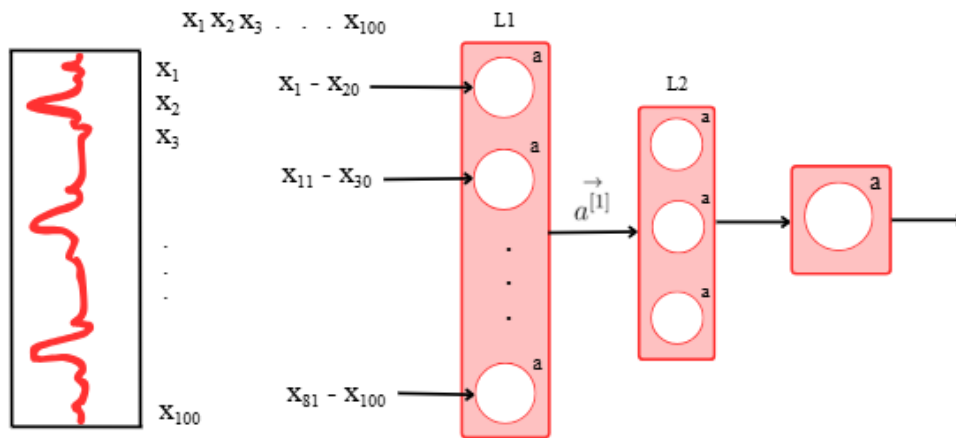
- **Convolutional:** Only select outputs from a previous layer go into each input neuron of the subsequent layer. (This is usually determined based on some application)

Example:



Note: This broken down view, results in faster computation and needs less training data (less prone to overfitting as well).

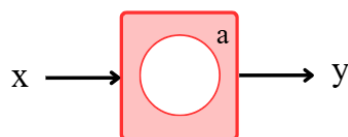
Example: EKG/ECG Signal



Computation Graphs

Example:

Consider a small Neural Network example again,

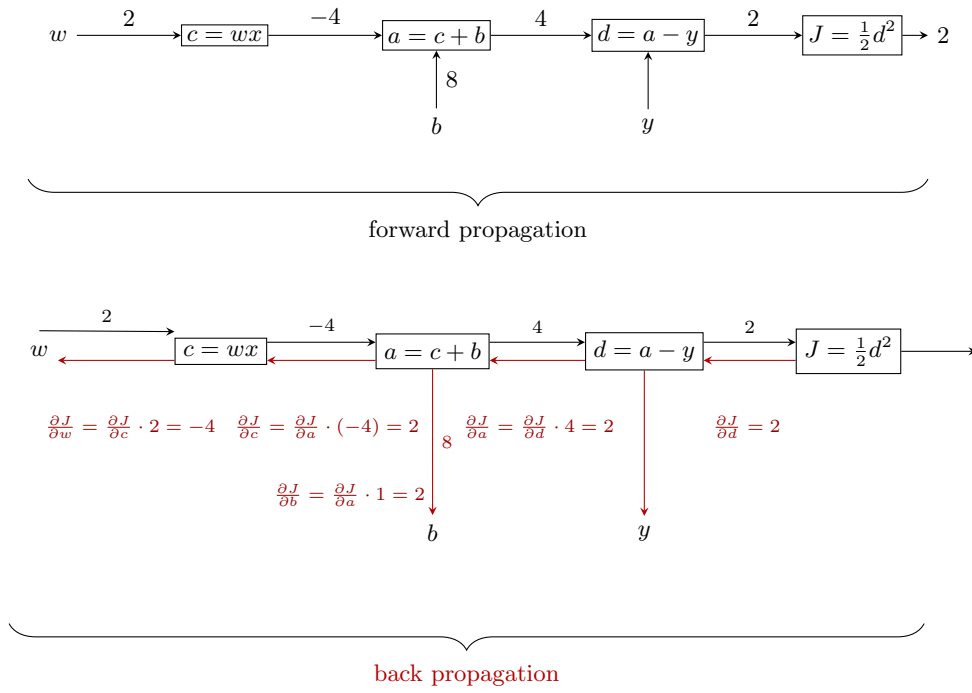


Assume, that this is activated with the linear activation function and has the associated cost function,

$$a = g(z) = wx + b, \quad J(w, b) = \frac{1}{2}(a - y)^2$$

Say $x = -2$ (input), $y = 2$ (output), $w = 2$ and $b = 8$ (parameters)

Computation Graph

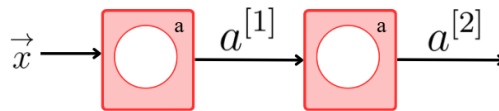


Note: Compute $\frac{\partial J}{\partial \alpha}$ once and use it to compute both $\frac{\partial J}{\partial w}$ and $\frac{\partial J}{\partial b}$. If there are N nodes and P parameters, compute parameters in roughly $(N + P)$ steps rather than $N * P$ steps

N	P	N + P	N * P
10,000	100,000	$1.1 * 10^5$	10^9

Example:

Consider a small but multilayer setup this time,

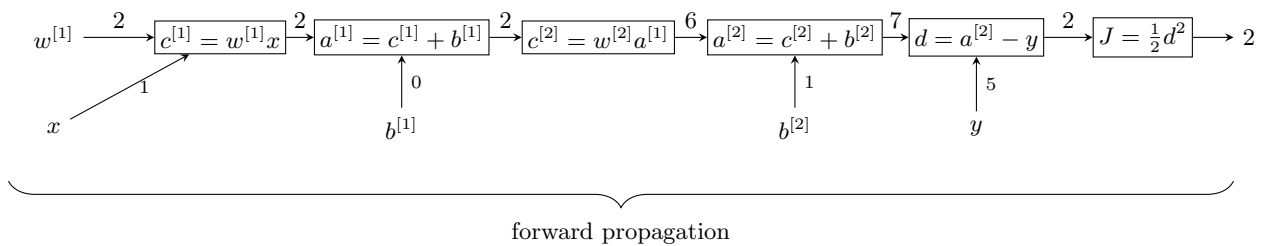


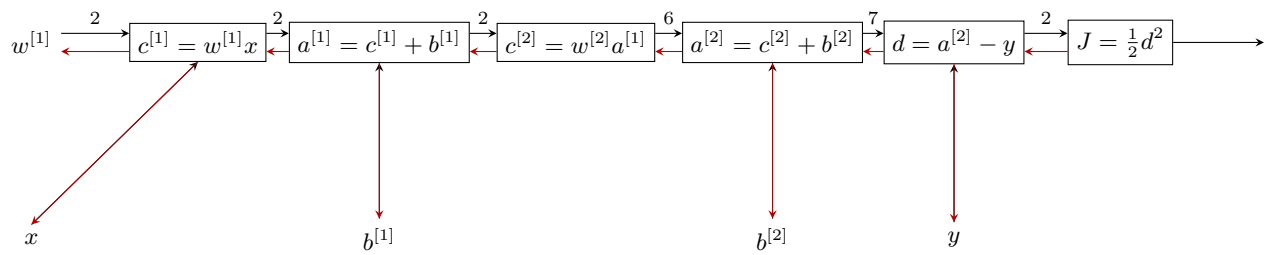
$$a^{[1]} = g(w^{[1]} \cdot x + b^{[1]}) = w^{[1]} \cdot x + b^{[1]} = 2.1 + 0 = 2,$$

$$a^{[2]} = g(w^{[2]} \cdot a^{[1]} + b^{[2]}) = w^{[2]} \cdot a^{[1]} + b^{[2]} = 3.2 + 1 = 7,$$

$$J(w, b) = \frac{1}{2}(a^{[2]} - y)^2 = \frac{1}{2}(7 - 5)^2 = 2$$

Computation Graph





back propagation

Gradient	Chain rule	Value
$\frac{\partial J}{\partial d}$	$= d = a^{[2]} - y = 7 - 5$	2
$\frac{\partial J}{\partial a^{[2]}}$	$= \frac{\partial J}{\partial d} \cdot 1 = 2 \times 1$	2
$\frac{\partial J}{\partial b^{[2]}}$	$= \frac{\partial J}{\partial a^{[2]}} \cdot 1 = 2 \times 1$	2
$\frac{\partial J}{\partial c^{[2]}}$	$= \frac{\partial J}{\partial a^{[2]}} \cdot 1 = 2 \times 1$	2
$\frac{\partial J}{\partial w^{[2]}}$	$= \frac{\partial J}{\partial c^{[2]}} \cdot a^{[1]} = 2 \times 2$	4
$\frac{\partial J}{\partial a^{[1]}}$	$= \frac{\partial J}{\partial c^{[2]}} \cdot w^{[2]} = 2 \times 3$	6
$\frac{\partial J}{\partial b^{[1]}}$	$= \frac{\partial J}{\partial a^{[1]}} \cdot 1 = 6 \times 1$	6
$\frac{\partial J}{\partial c^{[1]}}$	$= \frac{\partial J}{\partial a^{[1]}} \cdot 1 = 6 \times 1$	6
$\frac{\partial J}{\partial w^{[1]}}$	$= \frac{\partial J}{\partial c^{[1]}} \cdot x = 6 \times 1$	6

Debugging Tips

Example:

If we have implemented a regularised linear regression algorithm on housing prices,

$$J(w, b) = \frac{1}{2m} \sum_{i=1}^m f_{\vec{w}, b}(\vec{x}^{[i]} - y^{[i]}) + \frac{\lambda}{2m} \sum_{j=1}^n w_j^2$$

But it makes unacceptably large errors in predictions. What do you try next ?

- More training examples ?
- Try smaller feature set ?
- Additional features ?
- Adding polynomial features ?
- Try higher/lower values of λ

Need ML diagnostic

These would be tests which reveal issues/guidance on how to improve the model ?

Example: Consider again the housing price model,

$$f_{\vec{w},b}(\vec{x}) = w_1x + w_2x^2 + \dots + w_nx^n + b$$

Tip 1: Split the Dataset into training/test (70% and 30 %)

size	price		
2104	400	$(x^{(1)}, y^{(1)})$	} training examples
.	.	$(x^{(2)}, y^{(2)})$	
$\vdots$	$\vdots$	$\vdots$	
.	.	$(x^{(m_{train})}, y^{(m_{train})})$	
1427	199	$(x_{test}^{(1)}, y_{test}^{(1)})$	} test examples
$\vdots$	$\vdots$	$\vdots$	
.	.	$(x^{(m_{test})}, y^{(m_{test})})$	

Note: m_{train} = number of training examples, m_{test} = number of test examples.

$$m = m_{test} + m_{train}$$

New Diagnostic Model for Regularised Linear Regression

Compute Test Error:

$$J_{test}(\vec{w}, b) = \frac{1}{2m_{test}} \left[\sum_{i=1}^{m_{test}} \left(f_{\vec{w},b}(x_{test}^{(i)}) - y_{test}^{(i)} \right)^2 \right] + \cancel{\sum_{j=1}^n w_j^2}$$

Compute Training Error:

$$J_{train}(\vec{w}, b) = \frac{1}{2m_{train}} \left[\sum_{i=1}^{m_{train}} \left(f_{\vec{w},b}(\vec{x}_{train}^{(i)}) - \tilde{y}_{train}^{(i)} \right)^2 \right] + \cancel{\sum_{j=1}^n w_j^2}$$

Note: Regularisation term is not included.

Note: When J_{test} is higher (much) than J_{train} we can say that the data might have over-fit (could benefit from variety and simpler structure or other ways to generalise better).

Train / Test for Regularised Logistic Regression

Usual Loss Function

$$J(\vec{w}, b) = -\frac{1}{m_{train}} \sum_{i=1}^{m_{train}} \left[y^{(i)} \log(f_{\vec{w},b}(\vec{x}^{(i)})) + (1 - y^{(i)}) \log(1 - f_{\vec{w},b}(\vec{x}^{(i)})) \right] + \frac{\lambda}{2m_{train}} \sum_{i=1}^n w_j$$

Compute Test Error

$$J(\vec{w}, b) = -\frac{1}{m_{test}} \sum_{i=1}^{m_{test}} \left[y_{test}^{(i)} \log(f_{\vec{w},b}(\vec{x}_{test}^{(i)})) + (1 - y_{test}^{(i)}) \log(1 - f_{\vec{w},b}(\vec{x}_{test}^{(i)})) \right]$$

Compute Train Error

$$J(\vec{w}, b) = -\frac{1}{m_{train}} \sum_{i=1}^{m_{train}} \left[y_{train}^{(i)} \log(f_{\vec{w},b}(\vec{x}_{train}^{(i)})) + (1 - y_{train}^{(i)}) \log(1 - f_{\vec{w},b}(\vec{x}_{train}^{(i)})) \right]$$

Another Procedure for Classification tasks

Measure fraction of the test set and the fraction of the train set that the algorithm has misclassified:

$$\hat{y} = \begin{cases} 1 & \text{if } : f_{\vec{w},b}(\vec{x}^{(i)}) \geq 0.5 \\ 0 & \text{if } : f_{\vec{w},b}(\vec{x}^{(i)}) < 0.5 \end{cases}$$

We can count the number of instances where $\hat{y} \neq y$

Note: $J_{test}(\vec{w}, b)$ is the fraction of the test set that has been misclassified

Note: $J_{train}(\vec{w}, b)$ is the fraction of the train set that has been misclassified

Model Selection

Example:

$$\begin{aligned} d = 1 : f_{\vec{w},b}(\vec{x}) &= w_1x + b && \rightarrow w^{<1>}, b^{<1>} \rightarrow J_{test}(w^{<1>}, b^{<1>}) \\ d = 2 : f_{\vec{w},b}(\vec{x}) &= w_1x + w_2x^2 + b && \rightarrow w^{<2>}, b^{<2>} \rightarrow J_{test}(w^{<2>}, b^{<2>}) \\ d = 3 : f_{\vec{w},b}(\vec{x}) &= w_1x + w_2x^2 + w_3x^3 + b && \rightarrow w^{<3>}, b^{<3>} \rightarrow J_{test}(w^{<3>}, b^{<3>}) \\ &\vdots && \\ d = 10 : f_{\vec{w},b}(\vec{x}) &= w_1x + w_2x^2 + \dots + w_{10}x^{10} + b && \rightarrow w^{<10>}, b^{<10>} \rightarrow J_{test}(w^{<10>}, b^{<10>}) \end{aligned}$$

Say 5 does well, we choose $d = 5$ and we get the lowest values.

Note: We can ask ourselves how well does our model perform ? Report error $J_{test}(w^{<5>}, b^{<5>})$. Though $J_{test}(w^{<5>}, b^{<5>})$ is likely to be an optimistic estimate of generalisation error, i.e., $J_{test}(w^{<5>}, b^{<5>})$ (generalisation error). Because an extra parameter d (degree of polynomial) was choose using the test set.

Training / Cross-Validation / Test Set

Example:

Consider the housing example but with a new 3-way split instead.

size	price		
2104	400	$(x^{(1)}, y^{(1)})$	} training examples
.	.	$(x^{(2)}, y^{(2)})$	
$\vdots$	$\vdots$	$\vdots$	
.	.	$(x^{(m_{train})}, y^{(m_{train})})$	
.	.	$(x_{cv}^{(1)}, y_{cv}^{(1)})$	} cv examples
$\vdots$	$\vdots$	$\vdots$	
.	.	$(x^{(m_{cv})}, y^{(m_{cv})})$	
1427	199	$(x_{test}^{(1)}, y_{test}^{(1)})$	} test examples
$\vdots$	$\vdots$	$\vdots$	
.	.	$(x^{(m_{test})}, y^{(m_{test})})$	

Updated Diagnostic Model for Regularised Linear Regression

Compute Test Error:

$$J_{test}(\vec{w}, b) = \frac{1}{2m_{test}} \left[\sum_{i=1}^{m_{test}} \left(f_{\vec{w},b}(x_{test}^{(i)}) - y_{test}^{(i)} \right)^2 \right]$$

Compute Cross-Validation Error:

$$J_{cv}(\vec{w}, b) = \frac{1}{2m_{cv}} \left[\sum_{i=1}^{m_{cv}} \left(f_{\vec{w},b}(\vec{x}_{cv}^{(i)}) - \bar{y}_{cv}^{(i)} \right)^2 \right]$$

Compute Training Error:

$$J_{train}(\vec{w}, b) = \frac{1}{2m_{train}} \left[\sum_{i=1}^{m_{train}} \left(f_{\vec{w},b}(\vec{x}_{train}^{(i)}) - \bar{y}_{train}^{(i)} \right)^2 \right]$$

Model Selection

Example:

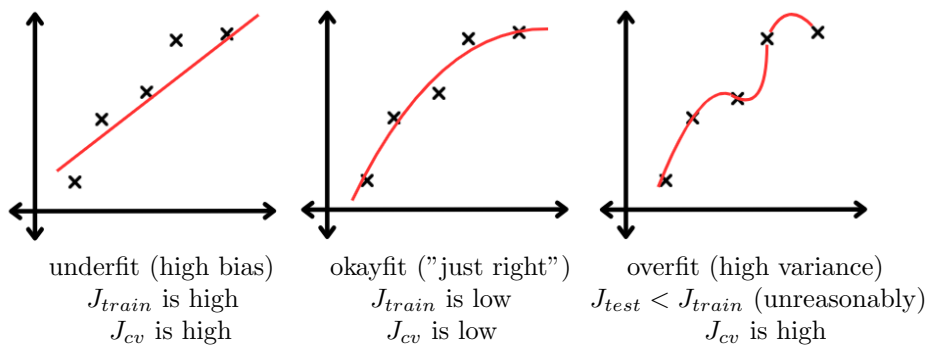
For the previous example we would pick $d = 4$,

$$w_1x + w_2x^2 + w_3x^3 + w_4x^4 + b \rightarrow J_{cv}(w^{<4>}, b^{<4>})$$

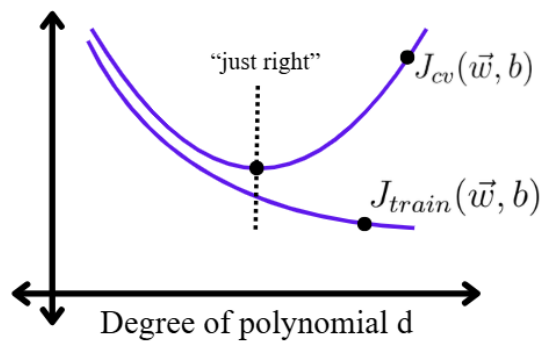
Estimate generalisation error using test set: $J_{test}(w^{<4>}, b^{<4>})$

Note: Even when choosing between Neural Network architectures, we pick the one with the lowest J_{cv} and estimate the generalisation error using the test $J_{test}(w^{<i>}, b^{<i>})$ where i is the numbered architecture for which the cross validation cost is the lowest.

Bias and Variance

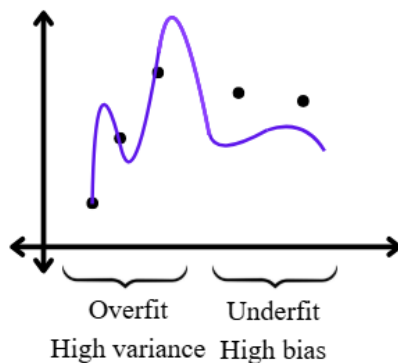


- **High bias (underfit):**
 J_{train} will be high, $J_{train} \approx J_{cv}$
- **High variance (overfit):**
 $J_{cv} \gg J_{train}$, J_{train} may be low.
- **High bias and high variance:**
 J_{train} will be high, and $J_{cv} \gg J_{train}$



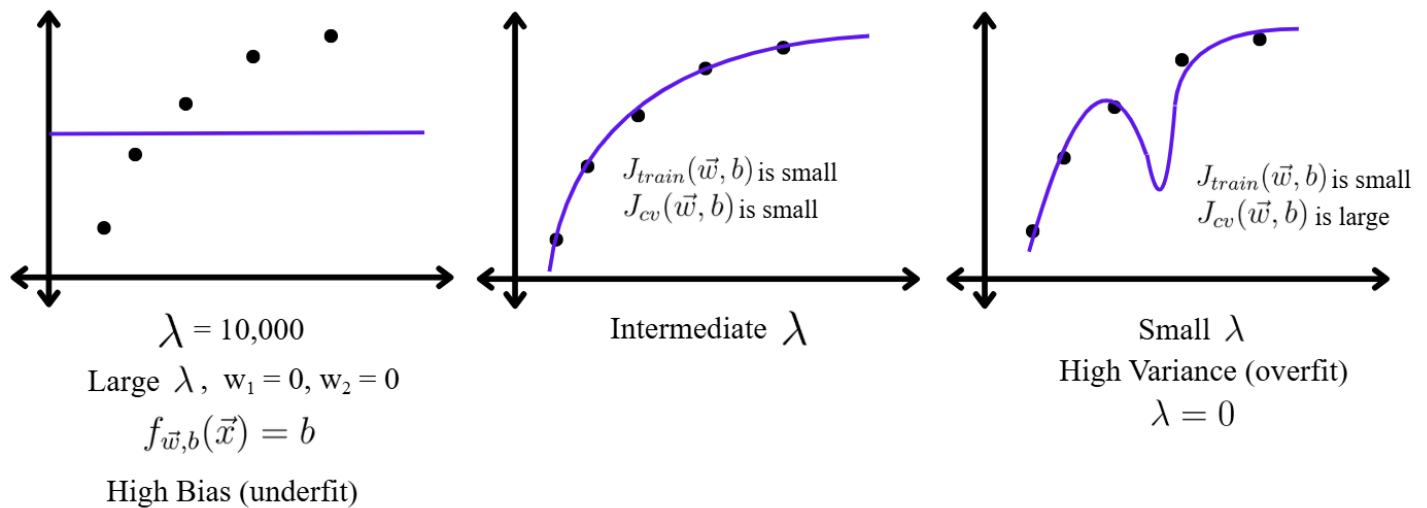
Example:

Example of a model having both high variance and high bias in the same model.



J_{train} will be high and $J_{cv} \gg J_{train}$

Choosing the Regularisation Parameter



Example:

$$\text{Model : } f_{\vec{w},b}(\vec{x}) = w_1 + w_2x^2 + w_3x^3 + w_4x^4 + b$$

We can try multiple values for regularisation parameter

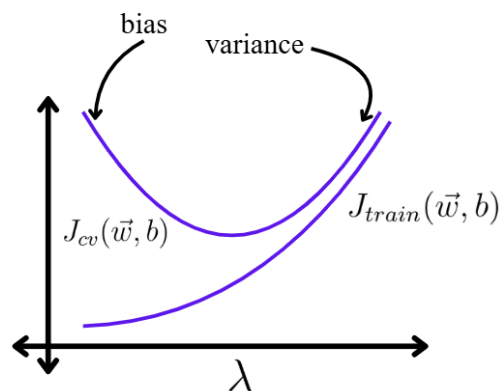
1. Try $\lambda = 0 \rightarrow \min_{\vec{w},b} J(\vec{w},b) \rightarrow w^{<1>}, b^{<1>} \rightarrow J_{cv}(w^{<1>}, b^{<1>})$
2. Try $\lambda = 0.01 \rightarrow \min_{\vec{w},b} J(\vec{w},b) \rightarrow w^{<2>}, b^{<2>} \rightarrow J_{cv}(w^{<2>}, b^{<2>})$
3. Try $\lambda = 0.02 \rightarrow \min_{\vec{w},b} J(\vec{w},b) \rightarrow w^{<3>}, b^{<3>} \rightarrow J_{cv}(w^{<3>}, b^{<3>})$
- $\vdots$
12. Try $\lambda = 10 \rightarrow \min_{\vec{w},b} J(\vec{w},b) \rightarrow w^{<12>}, b^{<12>} \rightarrow J_{cv}(w^{<12>}, b^{<12>})$

We report test error $J_{test}(w^{<5>}, b^{<5>})$ as we have picked 5, due to low value of $J_{cv}(w^{<5>}, b^{<5>})$

Example:

Bias and Variance as a function of Regularisation Parameter,

$$J(\vec{w}, b) = \frac{1}{2m} \sum_{i=1}^m \left(f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)} \right)^2 + \frac{\lambda}{2m} \sum_{j=1}^n w_j^2$$



Establishing a baseline level of performance

What is the level of error you can reasonably hope to get to:

- human-level performance
- competing algorithms' performance
- Guess based on experience

Example: Human speech recognition

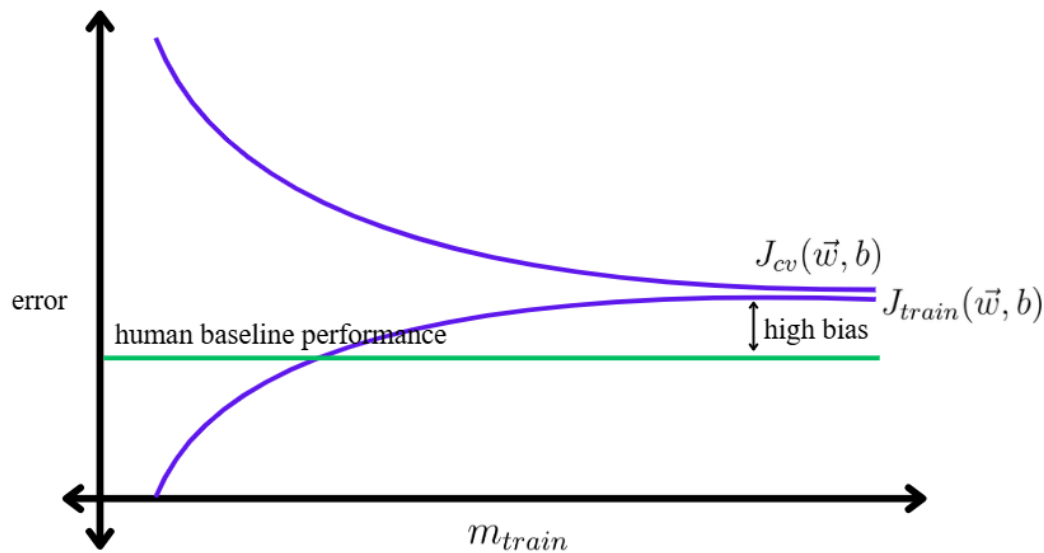
baseline performance	10.6%
training error (J_{train})	10.8%
CV error (J_{cv})	14.8%

Note: If the difference between baseline performance and J_{train} was large we would have said we have a high bias problem.

Note: If time is high we would conclude we have a high variance problem.

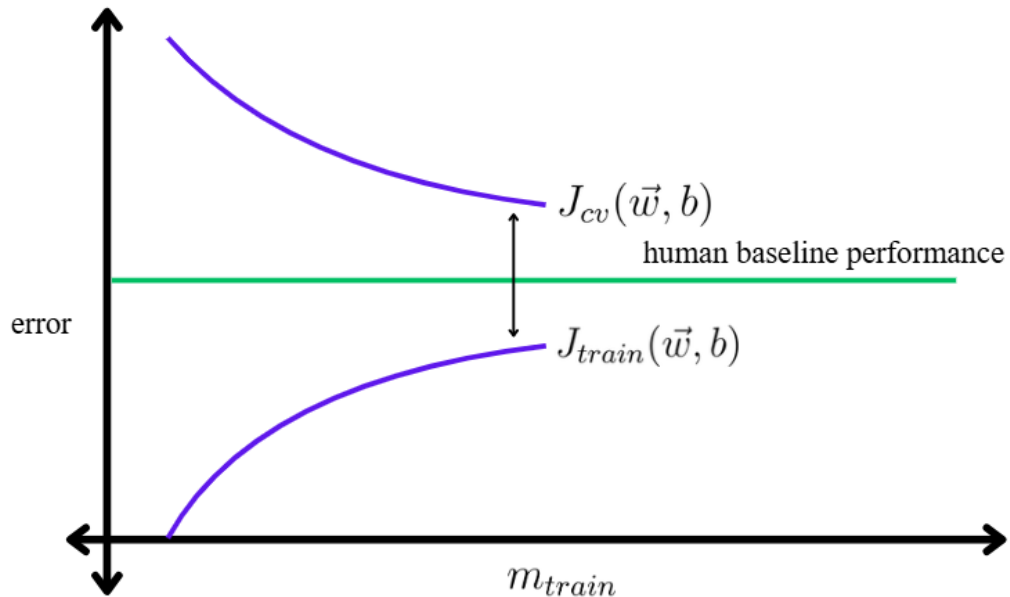
Example: High Bias case

$$\text{Model: } f_{\vec{w},b}(x) = w_1x + w_2x^2 + b$$



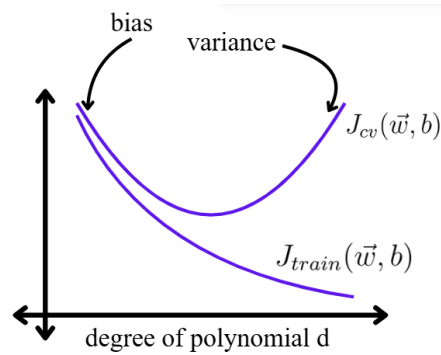
Note: For high bias cases, we would do better if we could get more training examples.

Example: High Variance case

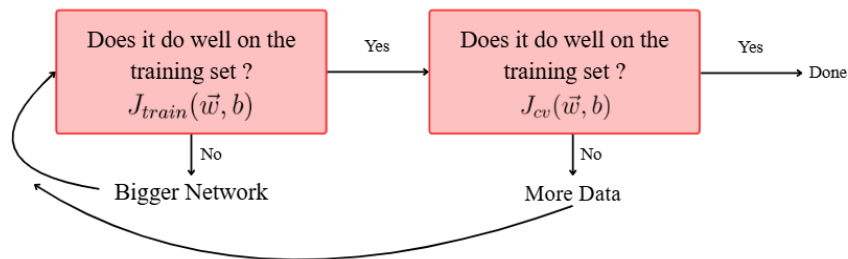


Tradeoff of Choice

$$\underbrace{f_{\vec{w},b}(x) = w_1x + b}_{\text{too simple model (high bias)}} \xleftrightarrow{\text{tradeoff}} \underbrace{f_{\vec{w},b}(x) = w_1x + w_2x^2 + w_3x^3 + w_4x^4 + b}_{\text{too complex model (high variance)}}$$

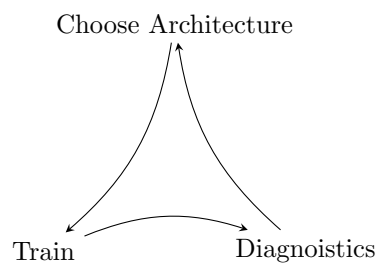


Note: Large Neural Network when trained over small / moderate datasets are low bias machines



Note: A large Neural Network will usually do as well or as better then a smaller one so long as regularisation is chosen appropriately.

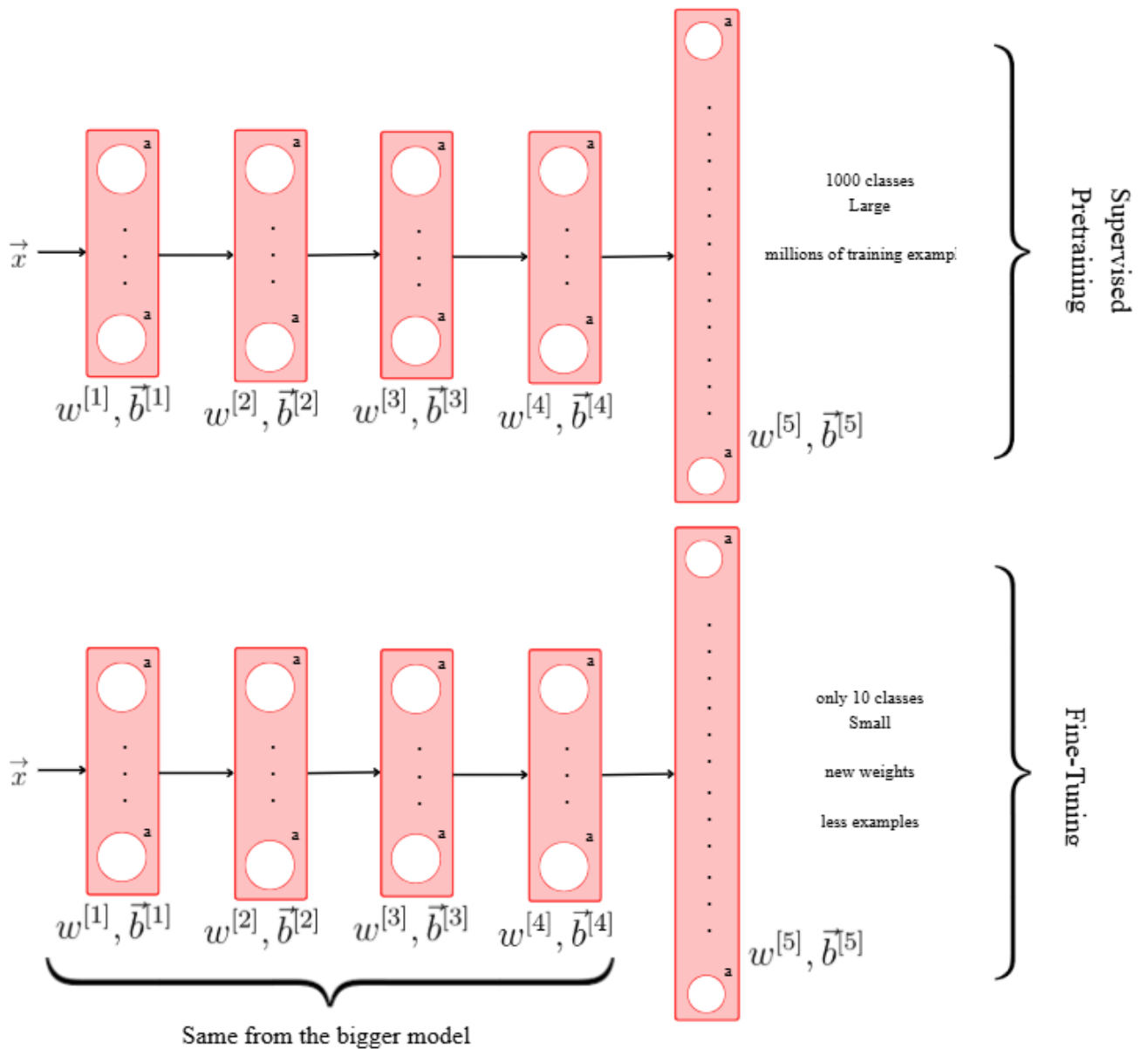
Iterative Loop for Machine Learning



Transfer Learning

Note: Useful when you don't have lots of data.

Example: Character recognition (0-9),



To train our model, (fine-tuning) we can either

- take / use the hidden-layer weights as-is
- Run propagation starting from the values of hidden-layer weights

This helps us train Neural Networks, on limited data for which high data might be available in a superset context.

Skewed Data and the Confusion Matrix

A lot of the times the dataset has class-imbalance

Example: Rare-Disease Classification ($y = 1$: present, $y = 0$: absent)

We find that you've got 1% error on test set (99% correct diagnosis), not very useful as only 0.5% in the real world might have it.

Confusion Matrix

		Predicted Class	
		Positive (1)	Negative (0)
Actual Class	Positive (1)	True Positive (15)	False Positive (5)
	Negative (0)	False Negative (10)	True Negative (70)

Precision

$$\text{precision} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Positives}} = \frac{TP}{TP + FP}$$

Note: Of all patients where we predicted $y = 1$ (disease present), what fraction actually has the rare-disease ?

Recall

$$\text{Recall} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Negatives}} = \frac{TP}{TP + FN}$$

Note: Of all the patients actually having the rare disease, what fraction did we correctly detect as having it ?

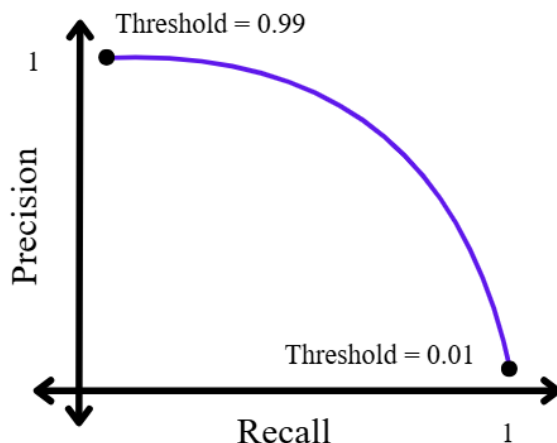
Example:

Logistic Regression: $0 < f_{\vec{w},b}(\vec{x}) < 1$

- predict 1 if $f_{\vec{w},b}(\vec{x}) \geq 0.5$
- predict 0 if $f_{\vec{w},b}(\vec{x}) < 0.5$

Note: One way of dealing with the rare-diseases is to raise the threshold from 0.5 to 0.7 (increases precision, but drops recall i.e., we are more correct but we don't diagnose as many patients).

Note: Suppose you didn't want to miss too many cases ? Then we would drop the threshold from 0.5 to 0.3 (drops precision, but increases recall i.e., you are more wrong, but you diagnose a greater total of cases)



F1-Score

Example:

	precision (P)	Recall (R)	Average (A)	F1 Score
Algorithm-1	0.5	0.4	0.45	0.444
Algorithm-2	0.7	0.1	0.4	0.175
Algorithm-3	0.02	1.0	0.501	0.0392

Note: Taking Average as we do in the 4th column is not very useful as it doesn't balance the trade-off, when either one of Recall (R) or Precision (P) is very very small ! For this reason we introduce the F1-score quantity

$$F1 - score = \frac{1}{\frac{1}{2} \left(\frac{1}{P} + \frac{1}{R} \right)} = 2 \frac{PR}{P + R}$$

Note: The Harmonic mean considers the possible disproportion of values and gives a good trade-off.

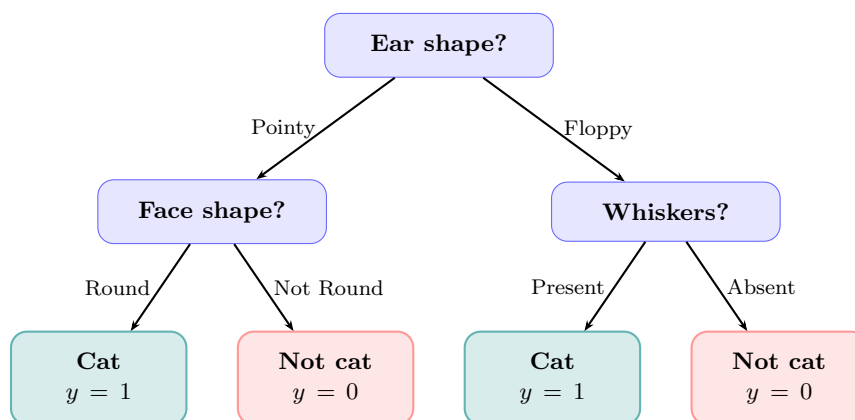
Decision Tree Model

Example:

Ear shape	Face shape	Whiskers	Cat
Pointy	Round	Present	1
Floppy	Not Round	Present	1
Floppy	Round	Absent	0
Pointy	Not Round	Present	0

$\underbrace{\hspace{10em}}_{\mathbf{x} \text{ (features)}} \quad \underbrace{\hspace{2em}}_{\mathbf{y} \text{ (binary)}}$

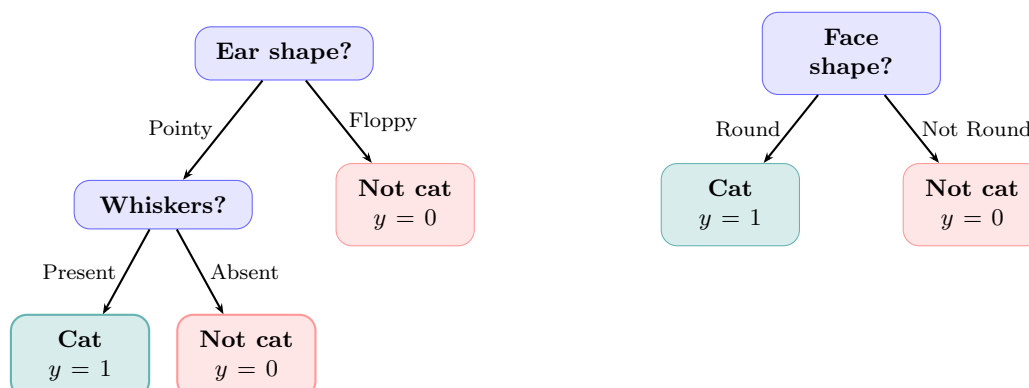
Note: Take note of how the value for each feature in x, are categorical, i.e., **discrete values** (non-continuous) and **finite**



Types of Nodes

- **root node:** Ear Shape ?
- **decision node:** Face Shape ? Whiskers ?
- **leaf node:** Cat, Not Cat

Note: There can be many different types of Decision Trees that can help us make decisions



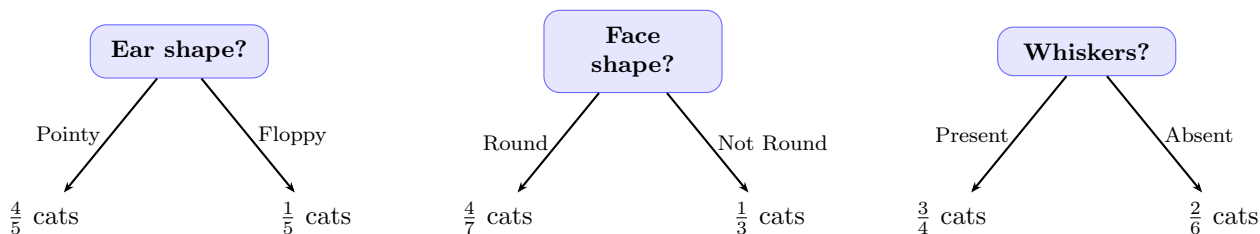
Note: Goal is to pick one which does well on data and generalises to CV/Test data.

Decision Tree Learning

Decision 1: How to choose what feature to split on at each node ?

- maximise purity

Example:



Decision 2: When do we stop splitting ?

- When a node is 100% class.
- When splitting a node will result in the tree exceeding a max depth.
- When improvements in purity score are below a threshold.
- When number of examples in a node are below a threshold.

Defining Purity

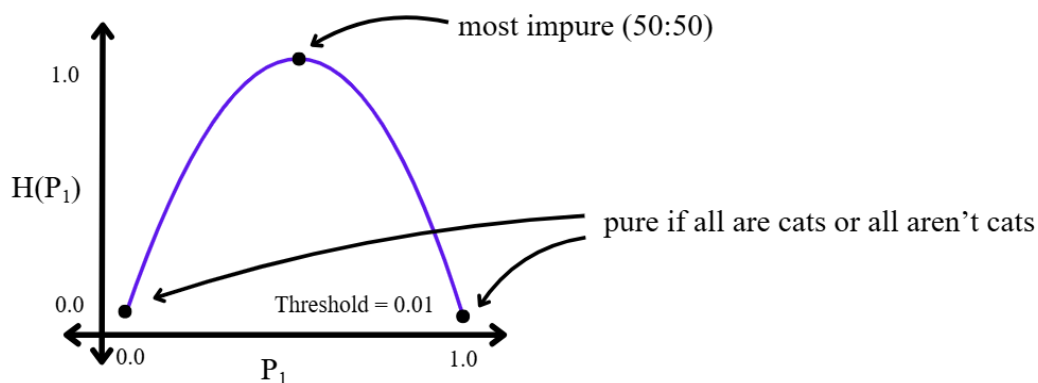
Entropy

Example:

Let

P_1 = fraction of examples that are cats:

Then we have the entropy function $H()$,



$$\because P_1 = 1 - P_0 \text{ or } P_0 + P_1 = 1$$

$$\therefore H(P_1) = -P_1 \log_2(P_1) - P_0 \log_2(P_0)$$

$$\implies H(P_1) = -P_1 \log_2 P_1 - (1 - P_1) \log_2 (1 - P_1)$$

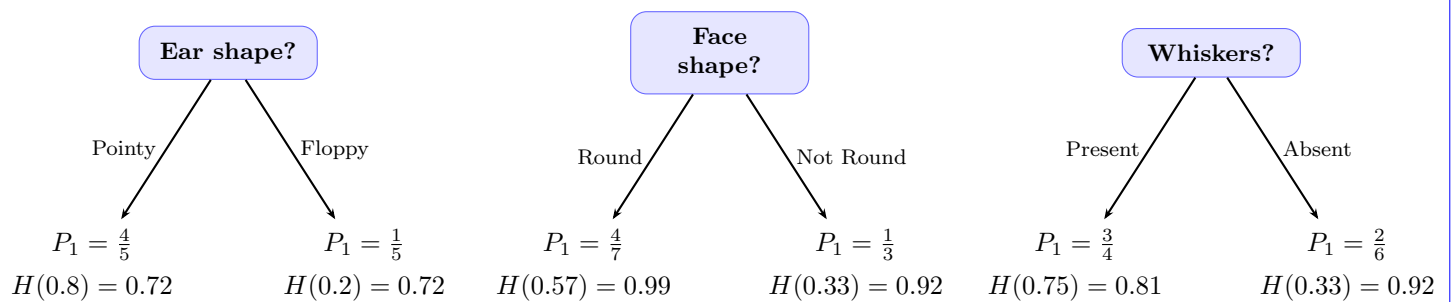
Note: We take $\log_2()$ instead of $\ln()$ to get apex at 1.

Note: By convention for computing Entropy we take:

$$0 \cdot \log_2 0 = 0$$

Choosing a split

Example:



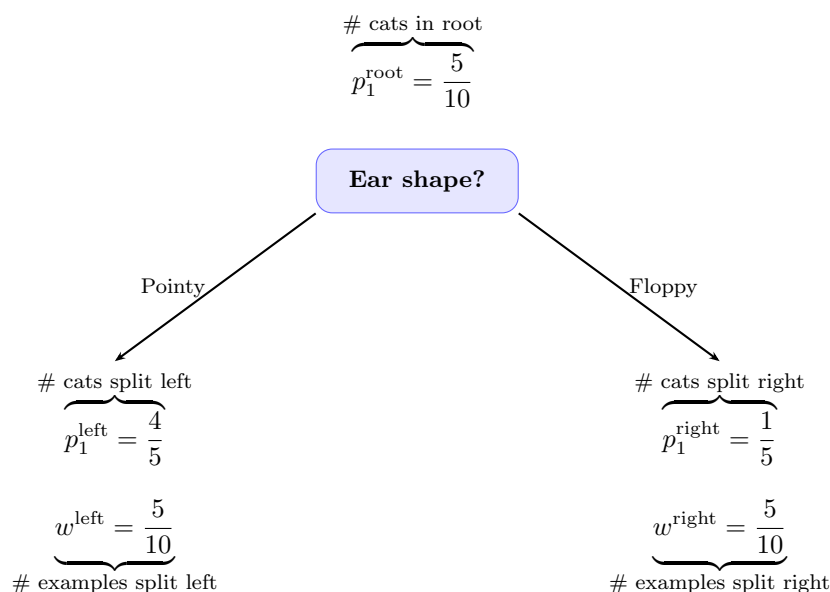
Note: It's better to take a weighted average of the entropy as $\frac{1}{3}$ and $\frac{1}{4}$ can mean different things.

Example:

$$\begin{aligned}
 & H(0.5) = 1 \\
 \Rightarrow & H(0.5) - \left(\frac{5}{10} H(0.8) + \frac{5}{10} H(0.2) \right), H(0.5) - \left(\frac{7}{10} H(0.57) + \frac{3}{10} H(0.33) \right), H(0.5) - \left(\frac{4}{10} H(0.75) + \frac{6}{10} H(0.33) \right) \\
 & \Rightarrow 0.28 \qquad \qquad \qquad = 0.03 \qquad \qquad \qquad = 0.12
 \end{aligned}$$

Note: These values measure the entropy reduction after the split, and is termed as **Information Gain**

Example:



Information Gain

$$= H(p_1^{\text{root}}) - (w^{\text{left}} H(p_1^{\text{left}}) + w^{\text{right}} H(p_1^{\text{right}}))$$

Decision Tree Algorithm

- Start with all examples at the root node.
- Calculate Information Gain for all possible features, and pick the ones with the highest Information Gain (IG).
- split the dataset according to selected features, and create L and R branches of tree
- Keep Repeating splitting process until stopping criteria is met:
 - When a node is 100% one class.
 - When splitting a node will result in the tree exceeding a maximum depth.

- Information Gain from additional splits is less than the thresholds.
- when number of examples in a node is below a threshold.

One-Hot Encoding

Example:

x_1 ear shape	x_2 face shape	x_3 whiskers	x_4 cat
pointy/floppy/oval			
$\underbrace{\hspace{2cm}}$ 3 possible values	$\underbrace{\hspace{2cm}}$ 2 possible values	$\underbrace{\hspace{2cm}}$ 2 possible values	

Note: For discrete-valued features we can use one-hot encoding which converts them into binary features making their treatment much easier, and in our case helping us create binary trees.

ear shape	pointy	flappy	oval	face shape	whiskers	cat
pointy	1	0	0	<i>Round</i> $\rightarrow 1$	<i>pointy</i> $\rightarrow 1$	
pointy	0	0	1	<i>NotRound</i> $\rightarrow 0$	<i>pointy</i> $\rightarrow 1$	
pointy	0	0	1	<i>Round</i> $\rightarrow 1$	<i>Notpointy</i> $\rightarrow 0$	
pointy	1	0	0	<i>NotRound</i> $\rightarrow 0$		
pointy	0	0	1			
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	

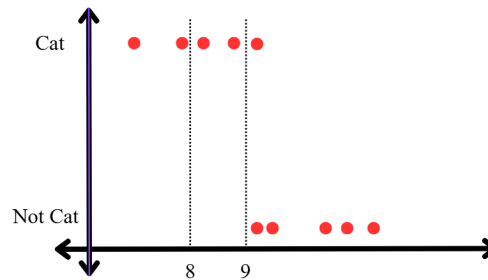
Note: If a categorised feature can take on k many values, then we can create out of that k many binary features (1/0).

Dealing with Continuous Data

Example:

ear shape	face shape	whiskers	weight (lbs)	cat
$\vdots$	$\vdots$	$\vdots$	7.2	
$\vdots$	$\vdots$	$\vdots$	8.8	
$\vdots$	$\vdots$	$\vdots$	15	

Note: Here weight is a continuous variable



Note: We can find the highest Information Gain that will give the best threshold

If we set $\text{weight} < 8$ lbs,

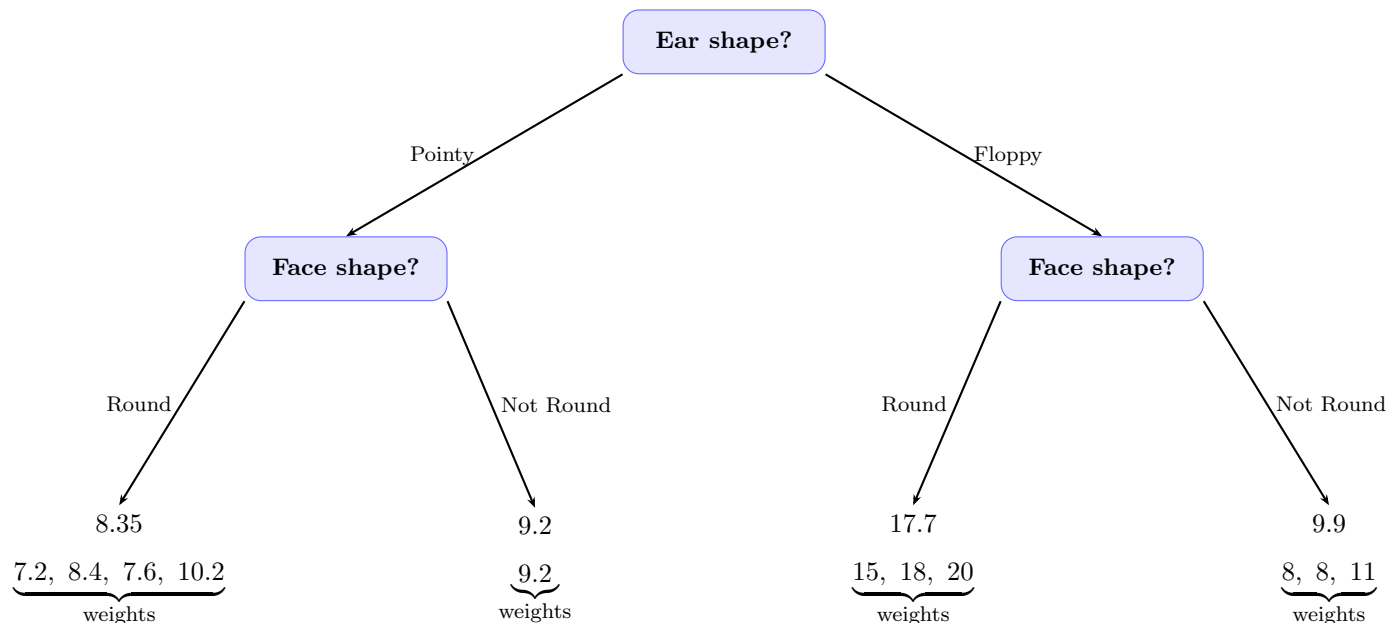
$$\text{Information Gain} = H(0.5) - \left(\frac{2}{10} H\left(\frac{2}{2}\right) + \frac{8}{10} H\left(\frac{3}{8}\right) \right) = 0.24$$

If we set $\text{weight} < 9$ lbs,

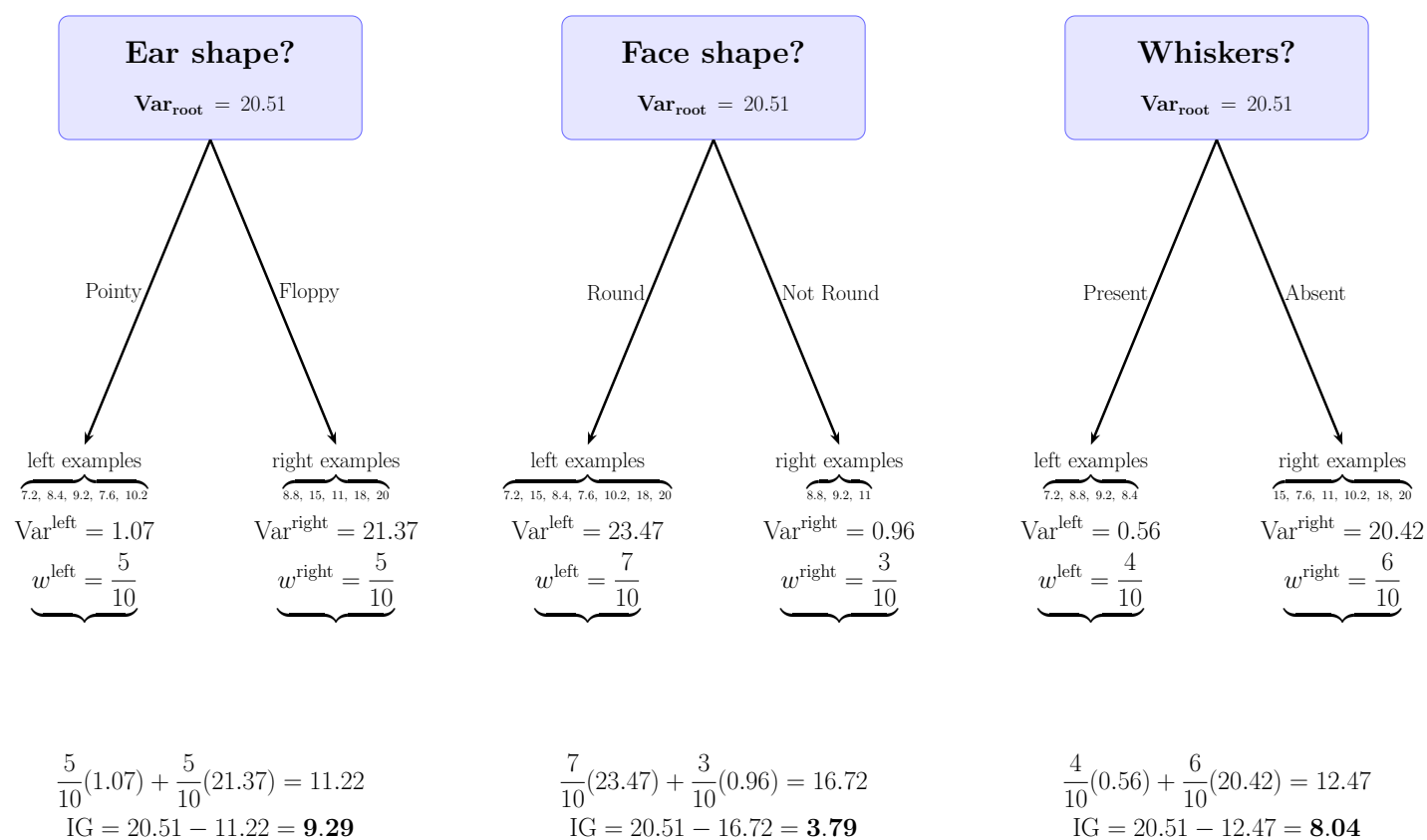
$$\text{Information Gain} = H(0.5) - \left(\frac{4}{10} H\left(\frac{4}{4}\right) + \frac{6}{10} H\left(\frac{1}{6}\right) \right) = 0.61$$

Regression with Decision Trees

Example:



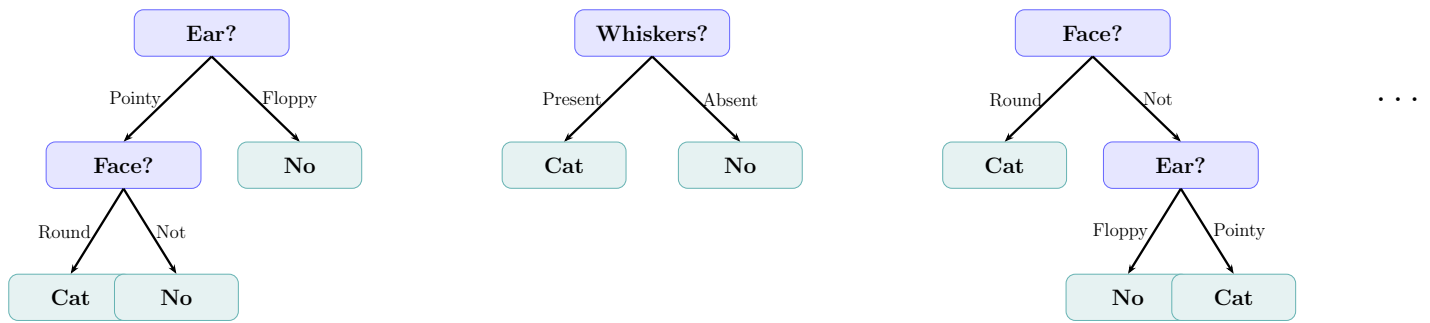
Choosing Split ?



Tree Ensembles

Note: Problem is one Decision Tree can be highly sensitive to small changes to data. Highest Information Gain feature might change the structure of the Decision Tree when new examples are added.

Example:



We can have a test example where we have a cat named Jerome (Pointy, Not Round, Present).

- T_1 predicts Not Cat
- T_2 predicts Cat
- T_3 predicts Cat

vote = majority(T_1, T_2, T_3) = "Cat"

Sampling with Replacement

This helps us create new training sets.

Example:

Given our 10 cat/not-cat training examples, we sample with replacement to build a new dataset of 10:

$$\{x^{(1)}, x^{(3)}, x^{(3)}, x^{(7)}, x^{(2)}, x^{(10)}, x^{(1)}, x^{(5)}, x^{(8)}, x^{(3)}\}$$

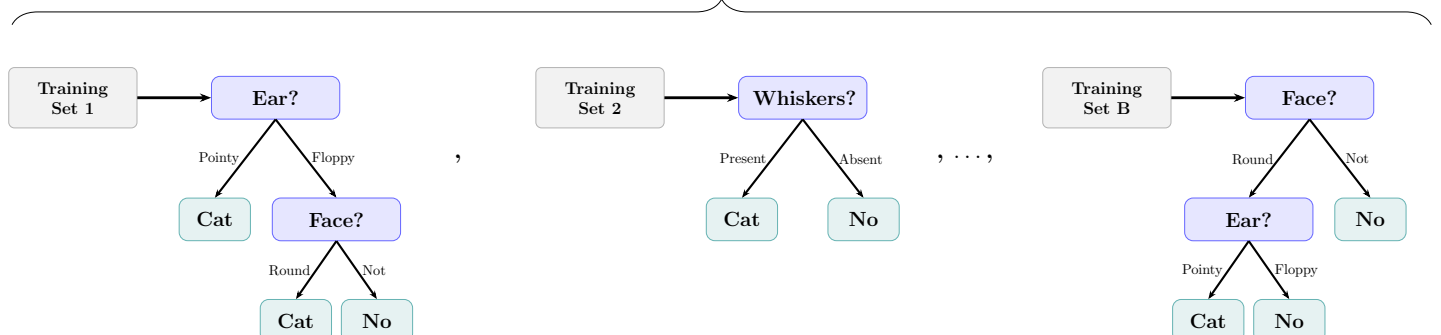
Notice $x^{(3)}$ appears 3 times and $x^{(4)}, x^{(6)}, x^{(9)}$ don't appear at all. We train a decision tree on this new bag, then repeat the process B times to get B different trees. Each tree sees a slightly different dataset $\Rightarrow$ different structure $\Rightarrow$ ensemble diversity.

Generating a Tree Sample

given a training set of size m :

- For $b = 1$ to B
 - Use sampling with replacement to create a new training set of size m .
 - Train a decision tree on the new dataset.

Bagged Decision Tree



Note: Even with Bagged Decision Tree we might end up with very similar root and root-adjacent nodes. We can randomise the feature choice as well.

Random Forest Algorithm

"At each node, when choosing a feature to use to split, if n features are available, pick a random subset of k ; n features and allow the algorithm to only choose from that subset of features."

Note: when k is large, k is often set to $\sqrt{n}$

Boosted Decision Trees

Note: Modify Bagged Tree Algorithm:

Given a training set of size m :

- For $b = 1$ to B :
 - Use sampling with replacement to create a new training set of size m .
 - But instead of picking from all examples with equal probability $\frac{1}{m}$, make it **more likely to sample misclassified examples** from previously trained trees.
 - Train a decision tree on the new dataset.

Note: The most commonly used version of this is **XGBoost** (**eXtreme Gradient Boosting**), which also has a built-in regularisation mechanism and is highly efficient in practice.

Decision Trees versus Neural Networks

DTs and Tree Ensembles

- Works well on tabular (structured) data
- Not recommended for unstructured data (images, audio, video, text)
- Fast to train
- Small DTs may be human interpretable (though interpretability is often overstated, most ensembles are extremely large and opaque)
- Works well on small to medium datasets
- Less prone to overfitting with ensembles (bagging, boosting)
- Cannot easily be fine-tuned on a pre-trained model

Neural Networks

- Works well on all types of data: tabular, images, audio, video, text
 - Recommended for unstructured data
 - Can be slow to train, especially large networks
 - Generally not interpretable (black box)
 - Scales better with very large datasets
 - Requires careful regularisation to avoid overfitting
 - Transfer learning possible, can fine-tune pre-trained networks
-